

INSTITUTO SUPERIOR POLITÉCNICO CÓRDOBA

TECNICATURA SUPERIOR EN

Desarrollo de Software

Desarrollo de Inteligencia Artificial

Unidad 2

Tema: Machine Learning introductorio

ÍNDICE

Concepto de aprendizaje automático..... 3
Definición operativa..... 3
Cómo “aprende” un modelo: la idea central..... 4
Paradigmas de aprendizaje..... 4
Generalización: el verdadero objetivo del entrenamiento..... 5
Regresión..... 6
¿Qué problema resuelve la regresión?..... 6
Regresión lineal simple..... 7
La función de costo: cómo se mide el error..... 7
Descenso por gradiente (Gradient Descent)..... 8
Regresión lineal múltiple..... 9
Evaluación de un modelo de regresión..... 10
Supuestos y limitaciones de la regresión lineal..... 11
De la regresión lineal a modelos más complejos..... 11
Clasificación..... 11
De predecir un número a predecir una categoría..... 11
La salida de un clasificador: clase y probabilidad..... 12
Árboles de decisión..... 13
La idea central..... 13
¿Cómo elige el algoritmo qué pregunta hacer primero?..... 13
Por qué los árboles de decisión son un buen punto de entrada..... 14
Limitaciones..... 14
Random Forest..... 15
La idea conceptual: “la sabiduría de las masas” aplicada a árboles..... 15
¿Cómo se logra que los árboles sean distintos entre sí?..... 15
El costo de esta mejora: se pierde interpretabilidad directa..... 16
Random Forest en la práctica: scikit-learn..... 17
Clustering con K-Means..... 18
Un cambio de paradigma: de supervisado a no supervisado..... 18
La idea de K-Means..... 18
¿Cómo se elige K?..... 19
K-Means en la práctica..... 20
Evaluación de modelos de clasificación..... 21
Por qué accuracy sola no alcanza..... 21
La matriz de confusión: la base de todo lo demás..... 21
Accuracy..... 22
Precision (Precisión)..... 22
Recall (Sensibilidad, Exhaustividad)..... 22

La tensión entre precision y recall..... 23
F1 Score: un solo número que balancea ambas..... 23
Evaluación en la práctica con scikit-learn..... 24
Síntesis: qué mirar según el problema..... 24
Dejar un modelo listo para implementar (serialización con pickle)..... 25
El problema que resuelve esta sección..... 25
Pickle: la herramienta estándar de Python..... 25
Ejemplo completo: de entrenar a dejar listo para producción.....27
Una nota importante de diseño: no todo lo que hay que guardar es el modelo... 28
Una advertencia de seguridad que hay que conocer..... 29
Glosario..... 30
Preguntas de repaso..... 32
Actividades..... 34
Bibliografía..... 35

Concepto de aprendizaje automático

Definición operativa

En la Unidad 1 se estableció una distinción general entre software tradicional (comportamiento definido por reglas explícitas) y sistemas de IA (comportamiento inferido a partir de datos). El aprendizaje automático (Machine Learning, ML) es la disciplina concreta que instrumenta esa segunda vía: es el subcampo de la IA que estudia algoritmos capaces de mejorar su desempeño en una tarea a partir de la experiencia (datos), sin ser programados explícitamente para esa tarea.

La definición formal más citada es la de Tom Mitchell (1997): un programa aprende de la experiencia E con respecto a una tarea T y una medida de desempeño P, si su desempeño en T, medido por P, mejora con la experiencia E.

Para un desarrollador, conviene traducir esto a los tres componentes que hay que identificar en cualquier problema de ML antes de escribir una sola línea de código:

- **Tarea (T)**: qué se quiere predecir o decidir (por ejemplo, predecir el tiempo de resolución de un ticket).
- **Experiencia (E)**: los datos históricos disponibles (tickets pasados, con su tiempo real de resolución).
- **Medida de desempeño (P)**: la métrica que cuantifica qué tan bien lo está haciendo el modelo (por ejemplo, el error promedio en días).

Esta traducción no es un formalismo vacío: en la práctica, la causa más común de que un proyecto de ML fracase no es un mal algoritmo, sino una mala definición de T, E o P desde el inicio.

Cómo “aprende” un modelo: la idea central

Un modelo de ML, en su forma más simple, es una función parametrizada: y = f(x; θ), donde x es la entrada (features), y es la salida, y θ (theta) es un conjunto de parámetros ajustables. “Entrenar” un modelo significa buscar, de forma automática, los valores de θ que hacen que f(x; θ) se aproxime lo mejor posible a los valores reales observados en los datos.

Ese proceso de búsqueda se apoya en dos piezas técnicas centrales, que reaparecen constantemente en toda la unidad:

- **Función de costo (o función de pérdida, loss function)**: una función que mide qué tan “equivocado” está el modelo para un conjunto de parámetros dado. Cuanto más alto el valor, peor la predicción.
- **Algoritmo de optimización**: el procedimiento que ajusta iterativamente los parámetros θ para minimizar la función de costo. El más usado en ML es el descenso por gradiente (gradient descent), que se ve en detalle en la sección de regresión.

Esta mecánica, definir un error y ajustar parámetros para reducirlo iterativamente, es el núcleo común a prácticamente todos los algoritmos de aprendizaje automático, desde una regresión lineal simple hasta una red neuronal con miles de millones de parámetros.

Paradigmas de aprendizaje

Existen tres grandes paradigmas, que se distinguen por el tipo de experiencia (E) disponible:

Aprendizaje supervisado: el dataset de entrenamiento incluye, para cada ejemplo, tanto las variables de entrada como la respuesta correcta (la etiqueta o label). El modelo aprende a mapear entradas a salidas conocidas. Es el paradigma más usado en la industria hoy, y es exactamente donde entra la

regresión, el tema central de esta unidad. Ejemplos de software: predecir si un pull request va a introducir un bug (a partir de PRs históricos etiquetados); predecir el tiempo de respuesta de un endpoint según la carga.

Aprendizaje no supervisado: el dataset no tiene etiquetas. El modelo busca estructura, patrones o agrupamientos en los datos por sí mismo. Ejemplos de software: agrupar usuarios de una aplicación según patrones de uso similares (clustering), sin que nadie haya definido de antemano los grupos; detección de anomalías en logs de sistema.

Aprendizaje por refuerzo (Reinforcement Learning): un agente aprende a tomar decisiones secuenciales interactuando con un entorno, recibiendo una recompensa (o penalización) según el resultado de sus acciones. Ejemplos de software: optimización del enrutamiento de tráfico en redes de datacenters; entrenamiento de agentes que juegan videojuegos o resuelven tareas de robótica; parte del proceso de alineación de los LLMs modernos (RLHF) usa aprendizaje por refuerzo.

Esta unidad se enfoca en el aprendizaje supervisado, porque la regresión, su algoritmo más representativo y didáctico, es la puerta de entrada estándar al razonamiento matemático que subyace a todo el aprendizaje automático.

Generalización: el verdadero objetivo del entrenamiento

Hay un punto conceptual central: el objetivo de entrenar un modelo no es que memorice los datos de entrenamiento, sino que generalice a datos nuevos que nunca vio.

Esto introduce dos fallas típicas:

- **Underfitting (subajuste)**: el modelo es demasiado simple para capturar el patrón real de los datos, y su error es alto incluso sobre los datos de entrenamiento.
- **Overfitting (sobreajuste)**: el modelo es tan flexible que termina ajustándose al ruido específico de los datos de entrenamiento en lugar de al patrón general, logrando un error bajísimo en entrenamiento pero un error alto sobre datos nuevos.

Por eso todo entrenamiento serio separa los datos en conjuntos de entrenamiento (para ajustar θ) y de prueba/test (para medir el desempeño real, sobre datos que el modelo no usó para aprender).

Regresión

¿Qué problema resuelve la regresión?

La regresión es la familia de algoritmos de aprendizaje supervisado cuyo objetivo es predecir una variable numérica continua a partir de una o más variables de entrada.

Ejemplos de preguntas que resuelve una regresión, en contextos de software e ingeniería:

- ¿Cuántos recursos de CPU va a consumir este servicio el próximo mes, dado su historial de uso?
- ¿Cuánto va a tardar en resolverse un ticket de soporte, dado el tipo de incidente y la carga actual del equipo?
- ¿Cuánto espacio de almacenamiento va a necesitar esta base de datos dentro de seis meses, según su tasa de crecimiento histórica?
- ¿Cuál va a ser la latencia promedio de un endpoint según la cantidad de requests concurrentes?

En todos los casos, la salida es un número real, no una etiqueta discreta, esa es la distinción central que separa a la regresión de la clasificación.

Regresión lineal simple

Es el caso más básico: una única variable de entrada (x) para predecir una variable de salida (y), asumiendo que la relación entre ambas puede aproximarse con una línea recta.

Modelo matemático:

$$\hat{y} = \beta_0 + \beta_1 x$$

Donde $\hat{y}$ (léase "y estimada") es la predicción del modelo, x es la variable de entrada (feature), β_0 (beta cero) es la ordenada al origen (intercept): el valor de $\hat{y}$ cuando x = 0, y β_1 (beta uno) es la pendiente (slope): cuánto cambia $\hat{y}$ por cada unidad que aumenta x.

Entrenar el modelo significa encontrar los valores de β_0 y β_1 que hacen que la recta se ajuste mejor a los datos observados.

Ejemplo de software: predecir el tiempo de build de un pipeline de CI (y, en minutos) en función de la cantidad de líneas de código modificadas en el commit (x). Si el modelo entrenado da $\beta_0 = 2$ y $\beta_1 = 0.01$, la interpretación directa es: un build "base" tarda 2 minutos, y cada línea de código modificada agrega, en promedio, 0.01 minutos adicionales.

La función de costo: cómo se mide el error

Para encontrar los mejores β_0 y β_1, primero hay que definir qué significa "mejor". La métrica estándar en regresión es el Error Cuadrático Medio (Mean Squared Error, MSE):

$$MSE = (1/n) * \Sigma(y_i - \hat{y}_i)^2$$

Donde n es la cantidad de observaciones, y_i es el valor real, y $\hat{y}_i$ es el valor predicho por el modelo para esa observación.

Dos decisiones de diseño están incorporadas en esta fórmula: se eleva al cuadrado la diferencia (en lugar de usar el valor absoluto) porque hace que la función sea diferenciable en todo punto (necesario para el descenso por gradiente) y porque penaliza de forma desproporcionada los errores grandes por sobre los pequeños (un error de 10 pesa 100 veces más que un error de 1, no 10 veces más). Y se promedia sobre todas las observaciones (1/n) para que la métrica no dependa del tamaño del dataset.

Entrenar el modelo, entonces, se reduce formalmente a un problema de optimización: encontrar los valores de β_0 y β_1 que minimizan el MSE.

Descenso por gradiente (Gradient Descent)

Este es el algoritmo que resuelve ese problema de optimización de forma iterativa, y es el mecanismo de entrenamiento más importante de todo el aprendizaje automático moderno, la misma idea, escalada a millones de parámetros, es lo que entrena las redes neuronales y los LLMs.

La intuición: imaginemos el MSE como una superficie (un “paisaje” con valles y colinas) en función de los valores posibles de β_0 y β_1. El objetivo es llegar al punto más bajo de ese paisaje. El descenso por gradiente funciona así:

1. Se inicializan β_0 y β_1 con valores arbitrarios (por ejemplo, cero, o aleatorios).
2. Se calcula el gradiente del MSE respecto de cada parámetro: la dirección en la que el error crece más rápido.
3. Se actualizan los parámetros dando un paso en la dirección opuesta al gradiente, de tamaño proporcional a una tasa de aprendizaje (learning rate, α):

```
β₀ ← β₀ - α * (∂MSE/∂β₀)
β₁ ← β₁ - α * (∂MSE/∂β₁)
```

4. Se repite el proceso miles de veces (cada repetición se llama época o iteración), hasta que el error deja de disminuir de forma significativa (el modelo “converge”).

La tasa de aprendizaje (α) es un parámetro crítico de diseño: si es demasiado alta, el algoritmo puede "saltar" el mínimo y no converger (oscila o diverge); si es demasiado baja, el entrenamiento converge, pero puede tardar muchísimas iteraciones (y mucho cómputo) en llegar a un buen resultado. Ajustar este tipo de parámetros, que no se aprenden de los datos, sino que se configuran antes de entrenar, es lo que se conoce como ajuste de hiperparámetros.

Regresión lineal múltiple

En la práctica, casi ningún problema real depende de una sola variable. La regresión lineal múltiple extiende el modelo simple para incorporar varias variables de entrada:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

Donde cada x_i es una variable (feature) distinta, y cada β_i es el peso o coeficiente que el modelo aprende para esa variable, cuantifica cuánto contribuye esa variable, de forma independiente, a la predicción final.

Ejemplo de software: predecir el tiempo de resolución de un ticket de soporte (y, en horas) en función de varias variables: x_1 = prioridad del ticket (numérica, 1 a 5), x_2 = cantidad de tickets abiertos actualmente en el equipo, x_3 = antigüedad del sistema afectado (en años). El modelo entrenado podría arrojar algo como:

$$\hat{y} = 1.5 - 0.8 \cdot x_1 + 0.3 \cdot x_2 + 0.2 \cdot x_3$$

La lectura de este resultado es directamente accionable para el negocio: por cada punto que sube la prioridad, el tiempo estimado de resolución baja 0.8 horas (coeficiente negativo); por cada ticket adicional abierto en el equipo, el tiempo estimado sube 0.3 horas (el equipo está más saturado); sistemas más antiguos agregan tiempo de resolución de forma consistente. Esta capacidad de interpretar

directamente los coeficientes es una de las grandes ventajas de la regresión lineal frente a modelos más complejos (como redes neuronales), que son mucho más difíciles de explicar.

Evaluación de un modelo de regresión

No alcanza con medir el error sobre los datos de entrenamiento: hay que evaluar sobre datos de test, nunca vistos por el modelo durante el ajuste de θ. Las métricas estándar para regresión son:

- **MSE (Error Cuadrático Medio)**: ya definido antes. Su unidad son las unidades de y al cuadrado, lo cual lo hace difícil de interpretar directamente.
- **RMSE (Raíz del Error Cuadrático Medio)**: la raíz cuadrada del MSE. Queda en las mismas unidades que y, por lo que es directamente interpretable.
- **MAE (Error Absoluto Medio)**: el promedio del valor absoluto de las diferencias entre y real y ŷ predicho. No penaliza desproporcionadamente los errores grandes, por lo que es más robusto ante outliers.
- **R^2 (Coeficiente de determinación)**: mide qué proporción de la variabilidad de y es explicada por el modelo, en una escala de 0 a 1. Un R^2 de 0.85 se interpreta como “el modelo explica el 85% de la variación observada en los datos”.

Criterio de diseño de software: la elección de qué métrica priorizar no es neutral. Si un error grande es mucho más costoso que varios errores pequeños, conviene priorizar RMSE. Si todos los errores son igual de costosos proporcionalmente, MAE da una imagen más robusta del desempeño típico.

Supuestos y limitaciones de la regresión lineal

- **Linealidad**: asume que la relación entre las variables de entrada y la salida es aproximadamente lineal. Si la relación real es curva, una regresión lineal va a ajustar mal.

- **Sensibilidad a outliers**: como el MSE eleva al cuadrado los errores, unos pocos valores atípicos pueden distorsionar fuertemente los coeficientes aprendidos.
- **Multicolinealidad**: cuando dos o más variables de entrada están fuertemente correlacionadas entre sí, el modelo tiene dificultad para separar el efecto individual de cada una.

De la regresión lineal a modelos más complejos

- **Regresión polinómica**: extiende la regresión lineal agregando potencias de las variables, para capturar relaciones no lineales.
- **Regresión regularizada (Ridge, Lasso)**: agrega una penalización a la función de costo para evitar que los coeficientes crezcan demasiado, reduciendo el riesgo de overfitting.
- **Árboles de decisión y Random Forest para regresión**: dividen el espacio de datos en regiones y predicen un valor promedio por región, a costa de perder la interpretabilidad directa de los coeficientes.
- **Redes neuronales para regresión**: capaces de aproximar relaciones arbitrariamente complejas, pagando el costo de menor explicabilidad e interpretabilidad.

Clasificación

De predecir un número a predecir una categoría

La regresión predice una variable numérica continua. La clasificación es la otra gran familia de algoritmos de aprendizaje supervisado, y resuelve un tipo de pregunta distinto: predecir a qué categoría (clase) pertenece una observación, entre un conjunto finito de opciones conocidas de antemano.

	Regresión	Clasificación
Qué predice	Un número continuo	Una categoría (clase)
Ejemplo	¿Cuánto va a tardar este ticket? (horas)	¿Este ticket es urgente o no urgente?
Salida típica	4.3, 12.8, 130.5	urgente / no urgente, o clase A / B / C

Ejemplos de preguntas de clasificación en software e ingeniería: ¿este pull request tiene alta probabilidad de introducir un bug, sí o no? (clasificación binaria); ¿este ticket de soporte corresponde a “bug”, “consulta” o “solicitud de feature”? (clasificación multiclase); ¿este correo es spam o no es spam?; ¿esta transacción es fraudulenta o legítima?.

La salida de un clasificador: clase y probabilidad

Un punto clave: la mayoría de los algoritmos de clasificación no producen únicamente una etiqueta, sino una probabilidad para cada clase posible, y la clase final se obtiene aplicando un umbral (por ejemplo, “si la probabilidad de fraude supera 0.5, clasificar como fraude”).

Esto es una decisión de diseño de software: mover ese umbral cambia el comportamiento del sistema sin tocar el modelo. Bajar el umbral de fraude de 0.5 a 0.3 hace que el sistema detecte más fraudes reales, pero también aumenta los falsos positivos. Ese balance se ve reflejado en dos métricas:

- **Precisión (precision)**: de todo lo que el modelo marcó como positivo, qué porcentaje era realmente positivo.
- **Sensibilidad o Recall**: de todos los casos positivos reales que existían, qué porcentaje logró detectar el modelo.

Ambas métricas suelen moverse en tensión, y qué priorizar depende del costo relativo de cada tipo de error en el problema concreto.

Árboles de decisión

La idea central

Un árbol de decisión es, probablemente, el algoritmo de clasificación más intuitivo que existe, porque su estructura es literalmente la misma que un diagrama de

flujo de decisiones: una secuencia de preguntas del tipo sí/no (o mayor/menor que), organizadas en forma de árbol, donde cada pregunta divide los datos en grupos cada vez más específicos, hasta llegar a una decisión final.

Ejemplo de software, clasificar si un pull request tiene riesgo alto o bajo de introducir un bug:

```
¿Modifica más de 200 líneas?
├── Sí → ¿Toca código de un módulo con historial de bugs?
│          ├── Sí → RIESGO ALTO
│          └── No → ¿Tiene tests asociados?
│                    ├── No → RIESGO ALTO
│                    └── Sí → RIESGO MEDIO
└── No → ¿Tiene tests asociados?
          ├── No → RIESGO MEDIO
          └── Sí → RIESGO BAJO
```

Cada pregunta es un nodo del árbol; cada respuesta posible es una rama; y cada resultado final es una hoja. Lo importante: el árbol no lo escribe una persona, el algoritmo lo aprende automáticamente a partir de los datos históricos, decidiendo por sí mismo cuáles son las preguntas más útiles y en qué orden hacerlas.

¿Cómo elige el algoritmo qué pregunta hacer primero?

A nivel introductorio, alcanza con la idea general: en cada paso, el algoritmo evalúa todas las preguntas posibles sobre las variables disponibles y elige la que logra separar mejor los datos en grupos lo más “puros” posible, es decir, grupos donde la mayoría de los casos pertenecen a una sola clase.

Esa noción de “pureza” se mide con métricas como la impureza de Gini o la entropía, que cuantifican qué tan mezcladas están las clases dentro de un grupo. El algoritmo repite este proceso de forma recursiva: elige la mejor pregunta, divide los datos, y vuelve a preguntarse cuál es la mejor siguiente pregunta dentro de cada grupo resultante, hasta que los grupos quedan suficientemente puros o se cumple algún criterio de corte.

Por qué los árboles de decisión son un buen punto de entrada

Los árboles de decisión son directamente interpretables. A diferencia de una red neuronal, donde es difícil explicar por qué se tomó una decisión puntual, un árbol de decisión se puede leer de arriba a abajo como una explicación en lenguaje natural: “este pull request se clasificó como riesgo alto porque modifica más de 200 líneas y toca un módulo con historial de bugs”. Esta es la razón por la cual los árboles de decisión siguen usándose hoy en contextos donde la explicabilidad es un requisito, como decisiones de crédito o de salud.

Limitaciones

- Un árbol de decisión entrenado sin ningún límite tiende a sobreajustarse: puede terminar creando preguntas tan específicas que memoriza el dataset de entrenamiento en lugar de aprender un patrón general.
- La forma habitual de controlar esto es limitar la profundidad máxima del árbol, un hiperparámetro que se configura antes de entrenar.
- Existe una evolución natural de este algoritmo, llamada Random Forest, que entrena muchos árboles de decisión distintos y combina sus resultados.

Random Forest

La idea conceptual: “la sabiduría de las masas” aplicada a árboles

Un árbol de decisión individual tiende a sobreajustarse, y las particularidades del dataset con el que se entrenó pueden dominar la predicción. Random Forest (bosque aleatorio) resuelve esto entrenando muchos árboles distintos y combinando sus predicciones.

La intuición es la misma que la de pedir una segunda, tercera y cuarta opinión antes de tomar una decisión importante: un árbol individual puede estar equivocado por haber visto una porción sesgada de los datos, pero es poco probable que cientos de árboles, entrenados de formas ligeramente distintas, estén todos equivocados de la misma manera. Al promediar sus respuestas, los errores individuales tienden a cancelarse.

Este principio general, combinar muchos modelos “débiles” o imperfectos para obtener un resultado colectivo más robusto, se conoce como ensamble (ensemble learning), y Random Forest es el ejemplo más representativo y más usado en la práctica.

¿Cómo se logra que los árboles sean distintos entre sí?

Random Forest introduce dos fuentes de aleatoriedad para forzar que cada árbol sea distinto:

- **Bootstrap (muestreo con reemplazo)**: cada árbol se entrena con una muestra aleatoria de los datos originales, elegida con reemplazo. Esto hace que cada árbol vea una versión ligeramente distinta del dataset.
- **Selección aleatoria de variables**: en cada división del árbol, el algoritmo no evalúa todas las variables disponibles, sino solo un subconjunto aleatorio de ellas. Esto evita que todos los árboles terminen dependiendo de las mismas variables “más fuertes”.

Con esas dos fuentes de aleatoriedad, se entrenan típicamente entre 100 y 500 árboles. La predicción final se obtiene combinando las predicciones individuales: en clasificación, por votación mayoritaria; en regresión, por promedio.

El costo de esta mejora: se pierde interpretabilidad directa

Un único árbol de decisión se puede leer y explicar como una secuencia de preguntas en lenguaje natural. Un Random Forest de 300 árboles, no, no hay una única secuencia de preguntas que explique la decisión, sino el resultado agregado de 300 secuencias distintas. Se gana precisión y estabilidad, se pierde la explicabilidad directa.

Como mitigación parcial, los Random Forest sí permiten calcular la importancia de las variables (feature importance): una medida de cuánto contribuyó, en promedio, cada variable a reducir la impureza a lo largo de todos los árboles del bosque.

Random Forest en la práctica: scikit-learn

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# Carga y preparación de datos
df = pd.read_csv("pull_requests.csv")

# Variables de entrada (features) y variable objetivo (target)
X = df[["lineas_modificadas", "archivos_modificados", "tiene_tests",
"modulo_con_historial_bugs"]]
y = df["riesgo_bug"]  # columna con las clases: "alto", "medio", "bajo"

# División en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Instanciar el modelo: 200 árboles, profundidad máxima 8 para evitar overfitting
modelo = RandomForestClassifier(
    n_estimators=200,
    max_depth=8,
    random_state=42
)

# Entrenamiento
modelo.fit(X_train, y_train)

# Predicción sobre datos nunca vistos
y_pred = modelo.predict(X_test)

# Evaluación
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

# Importancia de variables
importancias = pd.Series(modelo.feature_importances_, index=X.columns)
print(importancias.sort_values(ascending=False))
```

Observaciones sobre este código: `random_state=42` fija la semilla aleatoria para que el muestreo bootstrap y la selección de variables sean reproducibles entre ejecuciones. `test_size=0.2` implementa la separación train/test: un 80% de los datos para entrenar, un 20% reservado para evaluar de forma honesta. `max_depth=8` es el hiperparámetro que limita el sobreajuste, aplicado ahora a cada uno de los árboles del bosque.

Clustering con K-Means

Un cambio de paradigma: de supervisado a no supervisado

Todo lo visto hasta ahora, regresión, árboles de decisión, Random Forest, pertenece al aprendizaje supervisado: el dataset de entrenamiento incluye la respuesta correcta (y), y el modelo aprende a predecirla. K-Means es el primer algoritmo no supervisado de la unidad: el dataset no tiene etiquetas, y el objetivo no es predecir nada, sino descubrir estructura o agrupamientos naturales que ya existen en los datos.

Ejemplos de software: agrupar usuarios de una aplicación según su patrón de uso; agrupar servidores de una infraestructura según su perfil de consumo de recursos, para detectar configuraciones atípicas; agrupar tickets de soporte por similitud de contenido, como primer paso antes de decidir cómo clasificarlos.

La idea de K-Means

K-Means organiza los datos en K grupos (clusters), donde K es un número que se define de antemano. El algoritmo busca que los puntos dentro de un mismo grupo sean lo más parecidos posible entre sí, y lo más distintos posible de los puntos de otros grupos, midiendo “parecido” como distancia geométrica entre los valores numéricos de las variables.

El algoritmo, paso a paso:

1. **Inicialización**: se eligen K puntos al azar como “centros” iniciales de los clusters (centroides).

2. **Asignación**: cada dato del dataset se asigna al centroide más cercano (según distancia euclidiana).
3. **Actualización**: se recalcula la posición de cada centroide como el promedio de todos los puntos que quedaron asignados a él en el paso anterior.
4. **Repetición**: se repiten los pasos 2 y 3 hasta que los centroides dejan de moverse de forma significativa.

Ejemplo de software: agrupar servidores según dos variables, uso promedio de CPU (%) y uso promedio de memoria (%). Con K=3, el algoritmo podría converger a tres grupos: servidores de bajo consumo general, servidores con alto uso de CPU pero memoria normal, y servidores con alto uso de ambos recursos. Nadie definió estas categorías de antemano: emergieron de la estructura de los datos.

¿Cómo se elige K?

A diferencia de la clasificación, en K-Means el número de grupos es una decisión que hay que tomar antes de correr el algoritmo, y no siempre es obvio cuál es el valor correcto.

Una técnica introductoria estándar para elegirlo es el método del codo (elbow method): se corre K-Means para distintos valores de K, se mide para cada uno qué tan compactos quedan los clusters, y se grafica esa métrica en función de K. El punto donde la curva deja de mejorar significativamente (el “codo” del gráfico) es una buena referencia para elegir K.

K-Means en la práctica

```python
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# Datos de servidores: uso promedio de CPU y memoria
df = pd.read_csv("servidores.csv")
X = df[["uso_cpu_promedio", "uso_memoria_promedio"]]

# Estandarización: K-Means usa distancias geométricas, así que las
# variables deben estar en escalas comparables
X_escalado = StandardScaler().fit_transform(X)

# Método del codo para elegir K
inercias = []
for k in range(1, 8):
    modelo_k = KMeans(n_clusters=k, random_state=42, n_init=10)
    modelo_k.fit(X_escalado)
    inercias.append(modelo_k.inertia_)

plt.plot(range(1, 8), inercias, marker="o")
plt.xlabel("Cantidad de clusters (K)")
plt.ylabel("Inercia (compactación de los grupos)")
plt.title("Método del codo")
plt.show()

# Entrenamiento con el K elegido (por ejemplo, K=3 según el gráfico anterior)
modelo = KMeans(n_clusters=3, random_state=42, n_init=10)
df["cluster"] = modelo.fit_predict(X_escalado)

# Visualización de los grupos resultantes
plt.scatter(df["uso_cpu_promedio"], df["uso_memoria_promedio"], c=df["cluster"],
cmap="viridis")
plt.xlabel("Uso de CPU (%)")
plt.ylabel("Uso de memoria (%)")
plt.title("Servidores agrupados por perfil de consumo")
plt.show()
```

Dos observaciones: la estandarización previa (`StandardScaler`) es indispensable acá, más incluso que en regresión: si CPU se mide en 0-100% y memoria en gigabytes, la variable con números más grandes dominaría por completo el cálculo de distancias, distorsionando los grupos. Y a diferencia de todo lo anterior en esta unidad, acá no hay `y_test` ni métricas como accuracy o R^2, porque no hay una respuesta “correcta” contra la cual comparar. La evaluación de un clustering es, en general, más subjetiva e interpretativa que la de un modelo supervisado,

muchas veces la validación final la hace una persona del dominio, revisando si los grupos resultantes tienen sentido de negocio.

Evaluación de modelos de clasificación

Por qué accuracy sola no alcanza

La métrica más obvia para evaluar un clasificador, el porcentaje de aciertos, o accuracy, puede ser profundamente engañosa.

Ejemplo: un modelo que detecta transacciones fraudulentas, sobre un dataset donde el 99% de las transacciones son legítimas y solo el 1% son fraude. Un modelo "tonto" que siempre predice "no es fraude", sin analizar nada, va a tener un accuracy del 99%, y sin embargo es completamente inútil, porque no detecta ni un solo fraude real. Esta es la razón por la cual se necesita un conjunto de métricas más rico que un simple porcentaje de aciertos.

La matriz de confusión: la base de todo lo demás

Todas las métricas que siguen se derivan de una misma tabla, la matriz de confusión, que cruza lo que el modelo predijo contra lo que realmente ocurrió.

	Predicho: Positivo	Predicho: Negativo
Real: Positivo	Verdadero Positivo (VP)	Falso Negativo (FN)
Real: Negativo	Falso Positivo (FP)	Verdadero Negativo (VN)

- **Verdadero Positivo (VP)**: el modelo predijo "fraude" y realmente era fraude. Acierto.
- **Verdadero Negativo (VN)**: el modelo predijo "no fraude" y realmente no era fraude. Acierto.

- **Falso Positivo (FP)**: el modelo predijo “fraude” pero la transacción era legítima. Error: se bloquea a un cliente real sin motivo.
- **Falso Negativo (FN)**: el modelo predijo “no fraude” pero en realidad sí lo era. Error: se deja pasar un fraude real, típicamente el error más costoso en este dominio.

Accuracy

```
Accuracy = (VP + VN) / (VP + VN + FP + FN)
```

Es la proporción total de predicciones correctas sobre el total de casos. Es una métrica razonable sólo cuando las clases están relativamente balanceadas; con clases desbalanceadas, puede ocultar por completo un mal desempeño en la clase minoritaria.

Precision (Precisión)

```
Precision = VP / (VP + FP)
```

Responde a la pregunta: de todo lo que el modelo marcó como positivo, ¿qué porcentaje lo era realmente? Es una medida de qué tan confiable es una alerta positiva del modelo.

Ejemplo de software: en un sistema que marca automáticamente pull requests como “riesgo alto de bug”, una precisión baja significa que el equipo va a perder tiempo revisando manualmente muchos PRs que en realidad no tenían ningún problema, esto erosiona la confianza del equipo en la herramienta.

Recall (Sensibilidad, Exhaustividad)

```
Recall = VP / (VP + FN)
```

Responde a la pregunta complementaria: de todos los casos positivos que realmente existían, ¿qué porcentaje logró detectar el modelo? Es una medida de

qué tan completo es el modelo a la hora de encontrar todos los casos que le interesan.

Ejemplo de software: un recall bajo significa que hay PRs realmente riesgosos que están pasando sin ninguna alerta, el peor escenario posible para una herramienta de este tipo.

La tensión entre precision y recall

Estas dos métricas suelen moverse en direcciones opuestas, y el motivo es directo: ambas dependen del umbral de decisión que convierte una probabilidad en una etiqueta. Si se baja el umbral, el modelo se vuelve más “alarmista”: sube el recall, pero baja la precision. Si se sube el umbral, el modelo se vuelve más “conservador”: sube la precision, pero baja el recall.

Cuál priorizar es una decisión de negocio, no una decisión técnica: en un filtro de spam, un falso positivo suele ser peor que dejar pasar algún spam ocasional, por lo que se prioriza precision. En un sistema de detección de fallas críticas de infraestructura, dejar pasar una falla real suele ser mucho más costoso que investigar alguna falsa alarma, por lo que se prioriza recall.

F1 Score: un solo número que balancea ambas

```
F1 = 2 * (Precision * Recall) / (Precision + Recall)
```

Se usa la media armónica en lugar de un simple promedio porque castiga con más fuerza los casos donde una de las dos métricas es muy baja. El F1 Score es, en general, una buena métrica por defecto cuando las clases están desbalanceadas y no hay una razón clara para priorizar precision o recall por sobre la otra.

Evaluación en la práctica con scikit-learn

```python
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, ConfusionMatrixDisplay
)
import matplotlib.pyplot as plt

# y_test: valores reales; y_pred: predicciones del modelo (ej: Random Forest)

# Métricas individuales
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, pos_label="alto"))
print("Recall:", recall_score(y_test, y_pred, pos_label="alto"))
print("F1 Score:", f1_score(y_test, y_pred, pos_label="alto"))

# Matriz de confusión, en formato tabla
matriz = confusion_matrix(y_test, y_pred, labels=["alto", "bajo"])
print(matriz)

# Matriz de confusión, visualizada
disp = ConfusionMatrixDisplay(confusion_matrix=matriz, display_labels=["alto",
"bajo"])
disp.plot(cmap="Blues")
plt.title("Matriz de confusión - Riesgo de bug en PRs")
plt.show()
```

Un detalle práctico: `pos_label="alto"` indica explícitamente cuál es la clase "positiva" de interés, precision y recall no tienen sentido sin definir primero cuál es la clase que se está intentando detectar.

Síntesis: qué mirar según el problema

Métrica	Pregunta que responde	Cuándo priorizarla
Accuracy	¿Qué porcentaje total acertó?	Clases balanceadas, todos los errores igual de costosos
Precision	De lo que marqué como positivo, ¿cuánto era correcto?	El costo de un falso positivo es alto (ej: filtro de spam)
Recall	De todo lo positivo real, ¿cuánto detecté?	El costo de un falso negativo es alto (ej: detección de fallas críticas, fraude)

Métrica	Pregunta que responde	Cuándo priorizarla
F1 Score	¿Balance general entre precision y recall?	Clases desbalanceadas, sin razón clara para priorizar una sobre otra
Matriz de confusión	¿Dónde exactamente se equivoca el modelo?	Siempre, es la base para calcular todo lo anterior

Dejar un modelo listo para implementar (serialización con pickle)

El problema que resuelve esta sección

Hasta acá, todo el flujo de trabajo de la unidad ocurre dentro de una misma ejecución de Python: se corre un script o notebook, el modelo queda en memoria como una variable, y en cuanto termina la ejecución, ese modelo entrenado desaparece. Si mañana quisiéramos usarlo para predecir sobre un caso nuevo, habría que volver a entrenarlo desde cero, lo cual no tiene sentido en un sistema real: entrenar puede tardar minutos, horas o incluso días.

La solución es serializar el modelo: guardar en disco, en un archivo, el estado completo del objeto ya entrenado, para poder cargarlo después, en otro momento y hasta en otro programa, sin tener que reentrenar.

Pickle: la herramienta estándar de Python

Pickle es el módulo de la librería estándar de Python para serializar objetos: convierte prácticamente cualquier objeto de Python (incluyendo un modelo

entrenado de scikit-learn) en una secuencia de bytes que se puede guardar en un archivo, y que después se puede reconstruir exactamente igual a como estaba.

```python
import pickle

# --- Guardar (serializar) un objeto ya entrenado ---
with open("modelo.pkl", "wb") as archivo:
    pickle.dump(modelo, archivo)

# --- Cargar (deserializar) el objeto más tarde ---
with open("modelo.pkl", "rb") as archivo:
    modelo_cargado = pickle.load(archivo)
```

`"wb"` y `"rb"` significan write binary y read binary: pickle no genera texto legible, sino bytes, así que el archivo siempre se abre en modo binario. El objeto cargado es funcionalmente idéntico al original: se le puede llamar `.predict()` exactamente igual que si se acabara de entrenar en esa misma sesión.

Ejemplo completo: de entrenar a dejar listo para producción

```python
import pandas as pd
import pickle
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import f1_score

# 1. Entrenamiento
df = pd.read_csv("pull_requests.csv")
X = df[["lineas_modificadas", "archivos_modificados", "tiene_tests",
"modulo_con_historial_bugs"]]
y = df["riesgo_bug"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

modelo = RandomForestClassifier(n_estimators=200, max_depth=8, random_state=42)
modelo.fit(X_train, y_train)

# 2. Validación mínima antes de guardar: no tiene sentido serializar
#    un modelo que ni siquiera cumple un umbral de calidad aceptable
f1 = f1_score(y_test, modelo.predict(X_test), pos_label="alto")
print(f"F1 Score sobre test: {f1:.2f}")

if f1 < 0.70:
    raise ValueError("El modelo no alcanza el desempeño mínimo aceptable, no se
guarda.")

# 3. Serialización: el modelo queda listo como archivo
with open("modelo_riesgo_pr.pkl", "wb") as archivo:
    pickle.dump(modelo, archivo)

print("Modelo guardado en modelo_riesgo_pr.pkl")
```

Y en otro script completamente separado, por ejemplo el que correría dentro de un pipeline de CI, o detrás de un endpoint de una API interna, ya no hace falta nada de lo anterior:

```python
import pickle
import pandas as pd

# Cargar el modelo entrenado (sin necesidad de reentrenar nada)
with open("modelo_riesgo_pr.pkl", "rb") as archivo:
    modelo = pickle.load(archivo)

# Datos de un pull request nuevo, real, que llega para evaluar
pr_nuevo = pd.DataFrame([{
    "lineas_modificadas": 340,
    "archivos_modificados": 12,
    "tiene_tests": 0,
    "modulo_con_historial_bugs": 1
}])

prediccion = modelo.predict(pr_nuevo)
print("Riesgo estimado:", prediccion[0])
```

Este segundo bloque es, conceptualmente, la pieza que se integraría a un sistema real: podría ejecutarse dentro de un paso de un pipeline de CI/CD, detrás de un endpoint REST, o dentro de un job programado.

Una nota importante de diseño: no todo lo que hay que guardar es el modelo

Si en la preparación de datos se aplicó algún paso de transformación, por ejemplo la estandarización (`StandardScaler`) usada en K-Means, o una codificación de variables categóricas, ese transformador también fue “entrenado” con los datos, y hace falta guardarlo junto con el modelo, aplicando exactamente los mismos pasos sobre cualquier dato nuevo antes de predecir. Si no se hace así, se cae en el problema de training-serving skew: el modelo en producción recibe datos preparados de forma distinta a como fue entrenado, y su desempeño real se degrada sin que el código tire ningún error.

La forma simple de resolverlo es guardar ambos objetos juntos, por ejemplo en un diccionario:

```
paquete = {
    "modelo": modelo,
    "escalador": escalador,  # el StandardScaler ya ajustado con los datos de
entrenamiento
    "columnas": list(X.columns)  # para asegurar el mismo orden de variables al
predecir
}

with open("paquete_modelo.pkl", "wb") as archivo:
    pickle.dump(paquete, archivo)
```

Y al cargarlo, se reconstruyen los tres juntos, garantizando que la preparación de datos en producción sea idéntica a la del entrenamiento:

```
with open("paquete_modelo.pkl", "rb") as archivo:
    paquete = pickle.load(archivo)

modelo = paquete["modelo"]
escalador = paquete["escalador"]
columnas = paquete["columnas"]

datos_nuevos = datos_nuevos[columnas]          # mismo orden de columnas
datos_nuevos_escalados = escalador.transform(datos_nuevos)  # misma transformación
prediccion = modelo.predict(datos_nuevos_escalados)
```

Una advertencia de seguridad que hay que conocer

Pickle no es un formato seguro para cargar archivos de origen desconocido o no confiable: deserializar un archivo .pkl puede ejecutar código arbitrario en la máquina que lo carga, no es solo una advertencia teórica, es una técnica de ataque conocida. La recomendación práctica es simple: usar pickle libremente para modelos propios, generados y guardados por el propio equipo, pero nunca cargar un archivo .pkl recibido de una fuente externa o no verificada.

En un contexto de producción más exigente, se suelen usar alternativas como joblib (más eficiente para objetos con muchos arrays numéricos, aunque con la misma limitación de seguridad que pickle) o formatos de intercambio más controlados como ONNX.

Glosario

Aprendizaje no supervisado: paradigma de ML donde el dataset no tiene etiquetas, y el modelo busca estructura o patrones por sí mismo (ej: clustering).

Aprendizaje por refuerzo: paradigma de ML donde un agente aprende tomando decisiones secuenciales y recibiendo recompensas o penalizaciones según el resultado.

Aprendizaje supervisado: paradigma de ML donde el dataset incluye la respuesta correcta (etiqueta) para cada ejemplo, y el modelo aprende a predecirla.

Árbol de decisión: algoritmo de clasificación (o regresión) que estructura sus predicciones como una secuencia de preguntas organizadas en forma de árbol, aprendidas automáticamente a partir de los datos.

Bootstrap: técnica de muestreo con reemplazo usada en Random Forest para que cada árbol del bosque entrene con una versión distinta del dataset original.

Centroide: en K-Means, el punto que representa el centro de un cluster, calculado como el promedio de los puntos asignados a él.

Clasificación: tarea de aprendizaje supervisado cuyo objetivo es predecir una categoría (clase) entre un conjunto finito conocido de antemano.

Clustering: tarea de aprendizaje no supervisado que agrupa datos según su similitud, sin etiquetas previas.

Descenso por gradiente (Gradient Descent): algoritmo iterativo de optimización que ajusta los parámetros de un modelo en la dirección que reduce su función de costo.

Ensamble (Ensemble learning): técnica que combina las predicciones de varios modelos para obtener un resultado más robusto que el de cualquiera individualmente (ej: Random Forest).

F1 Score: métrica que combina precision y recall mediante su media armónica, útil cuando las clases están desbalanceadas.

Falso Negativo (FN): caso donde el modelo predijo la clase negativa, pero la clase real era positiva.

Falso Positivo (FP): caso donde el modelo predijo la clase positiva, pero la clase real era negativa.

Feature importance: medida de cuánto contribuye cada variable, en promedio, a las predicciones de un modelo (usada especialmente en Random Forest).

Función de costo (Loss function): función que cuantifica qué tan equivocadas son las predicciones de un modelo para un conjunto de parámetros dado.

Hiperparámetro: parámetro de configuración de un modelo que se define antes de entrenar (ej: tasa de aprendizaje, profundidad máxima de un árbol, cantidad de clusters K), a diferencia de los parámetros que el modelo aprende de los datos.

K-Means: algoritmo de clustering que agrupa los datos en K grupos, minimizando la distancia entre cada punto y el centroide de su grupo.

Matriz de confusión: tabla que cruza las predicciones de un clasificador contra los valores reales, distinguiendo verdaderos/falsos positivos y negativos.

MSE (Error Cuadrático Medio): métrica de error para regresión, que promedia el cuadrado de las diferencias entre valores reales y predichos.

Pickle: módulo de Python para serializar objetos (incluyendo modelos entrenados) en archivos, permitiendo guardarlos y reutilizarlos sin reentrenar.

Precision: de todo lo que un modelo predijo como positivo, la proporción que realmente lo era.

Random Forest: algoritmo de ensamble que entrena muchos árboles de decisión con aleatoriedad (bootstrap y selección de variables) y combina sus predicciones.

Recall (Sensibilidad): de todos los casos positivos reales, la proporción que el modelo logró detectar.

Regresión: tarea de aprendizaje supervisado cuyo objetivo es predecir una variable numérica continua.

Tasa de aprendizaje (Learning rate): hiperparámetro del descenso por gradiente que controla el tamaño de cada paso de ajuste de los parámetros.

Verdadero Negativo (VN): caso donde el modelo predijo la clase negativa y la clase real también era negativa.

Verdadero Positivo (VP): caso donde el modelo predijo la clase positiva y la clase real también era positiva.

Preguntas de repaso

Sobre el concepto de aprendizaje automático:

1. Según la definición de Mitchell, ¿cuáles son los tres componentes que hay que identificar en cualquier problema de ML? Dar un ejemplo propio.
2. ¿Cuál es la diferencia entre aprendizaje supervisado, no supervisado y por refuerzo? Dar un ejemplo de software para cada uno.
3. ¿Qué diferencia hay entre underfitting y overfitting, y por qué separar los datos en train/test ayuda a detectarlos?

Sobre regresión:

4. ¿Qué tipo de variable predice un modelo de regresión, y en qué se diferencia de lo que predice un clasificador?
5. ¿Para qué sirve la función de costo (MSE) y por qué se elevan al cuadrado las diferencias en lugar de usar el valor absoluto?
6. Explicar, con tus palabras, cómo funciona el descenso por gradiente. ¿Qué pasa si la tasa de aprendizaje es demasiado alta o demasiado baja?
7. ¿Por qué es más fácil interpretar los coeficientes de una regresión lineal que las predicciones de una red neuronal?

Sobre clasificación y árboles de decisión:

8. ¿Cómo decide un árbol de decisión cuál es la mejor pregunta para hacer en cada paso?
9. ¿Por qué se dice que un árbol de decisión es más interpretable que un Random Forest?
10. ¿Qué problema busca evitar limitar la profundidad máxima de un árbol?

Sobre Random Forest:

11. ¿Qué dos fuentes de aleatoriedad usa Random Forest para lograr que sus árboles sean distintos entre sí, y por qué es necesario que lo sean?
12. ¿Qué se gana y qué se pierde al pasar de un árbol de decisión individual a un Random Forest?

Sobre clustering con K-Means:

13. ¿En qué se diferencia el objetivo de K-Means del objetivo de un árbol de decisión o una regresión?
14. ¿Por qué es necesario estandarizar las variables antes de aplicar K-Means?
15. ¿Para qué sirve el método del codo?

Sobre evaluación de modelos:

16. ¿Por qué un accuracy del 99% puede ser engañoso en un problema de detección de fraude?
17. Explicar con un ejemplo propio la diferencia entre un falso positivo y un falso negativo.
18. ¿En qué situación conviene priorizar precision por sobre recall, y en cuál al revés? Dar un ejemplo de cada una.
19. ¿Por qué el F1 Score usa la media armónica en lugar de un promedio simple entre precision y recall?

Sobre dejar un modelo listo para implementar:

20. ¿Por qué no alcanza con guardar solamente el modelo entrenado si en la preparación de datos se usó, por ejemplo, un StandardScaler?

21. ¿Qué riesgo de seguridad tiene cargar un archivo .pkl de una fuente no confiable?

Actividades

Actividad 1 - De la Unidad 1 a la Unidad 2: ubicando los conceptos

En grupos, armar un cuadro que ubique cada algoritmo visto en esta unidad (regresión lineal, árbol de decisión, Random Forest, K-Means) según dos ejes: ¿es determinista o probabilístico? y ¿es supervisado o no supervisado? Justificar cada ubicación.

Actividad 2 - Diseñar un problema de regresión propio

Cada grupo debe proponer un problema de regresión en un contexto de software o de su interés, definiendo: la variable a predecir (y), al menos dos variables de entrada (x) razonables, y qué métrica (MSE, RMSE, MAE o R^2) usarían para evaluarlo y por qué.

Actividad 3 - Armar un árbol de decisión a mano

Analizar un dataset con 2-3 variables categóricas o numéricas simples y una clase objetivo (por ejemplo, aprobar o no aprobar una materia según horas de estudio y asistencia). Sin usar código, armar a mano un árbol de decisión razonable, eligiendo qué pregunta hacer primero y justificando por qué esa pregunta separa mejor los datos que otras alternativas.

Actividad 4 - Laboratorio con scikit-learn

Usando un dataset simple, completar el flujo completo visto en la unidad: cargar y preparar los datos, dividir en train/test, entrenar un RandomForestClassifier, calcular accuracy, precision, recall y F1 Score, y graficar la matriz de confusión. Guardar el modelo entrenado con pickle y, en un script

separado, cargarlo y usarlo para predecir sobre 2 o 3 casos nuevos inventados por el propio grupo.

Actividad 5 - Debate: ¿qué métrica importa más?

Consigna disparadora: se les presentan tres sistemas distintos (un filtro de spam, un sistema de diagnóstico médico asistido, y un sistema de recomendación de contenido) y deben discutir, para cada uno, si priorizarían precision o recall, y qué pasaría en la práctica si eligieran mal esa prioridad.

Bibliografía

Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3.ª ed.). O'Reilly Media.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2.ª ed.). Springer.

Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.

Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

Documentación oficial de scikit-learn: https://scikit-learn.org/stable/documentation.html

Documentación oficial del módulo pickle de Python: https://docs.python.org/3/library/pickle.html